Interview Answers - Allaudin Ansari

Interview Answers: Allaudin Ansari

Role: Data Scientist, Human Resocia (Japan) **Projects covered:** LexAI, AI-Powered Portfolio Optimization Engine, Real-Time E-Commerce Clickstream Lakehouse

Items in [square brackets] are personal details to fill in before the interview.

Part A: General Questions

Q1. Tell me about yourself and why you want to work in Japan.

Thank you for the opportunity. I'm Allaudin Ansari, a B.Tech Computer Science student at SRM Institute of Science and Technology, graduating in 2027, with a CGPA of 9.09 out of 10. My focus is applied data and AI work. I did a Quantum Optimization internship at Qclairvoyance Quantum Labs from December 2025 to January 2026, where I formulated molecular docking as QUBO problems and benchmarked solvers on a 675-variable instance. Outside the internship, I built three projects: LexAI, a retrieval-augmented question answering system; a portfolio optimization engine that uses reinforcement learning and MLOps; and a real-time clickstream lakehouse for e-commerce events. [Your personal reason for Japan, for example: what drew you to Japanese engineering teams or to AI work in manufacturing.] I'd like to work in an environment where engineering quality is taken seriously over a long period, and I'm committed to building my career there.

Q2. What is the difference between supervised and unsupervised learning? Give an example from a business.

Supervised learning uses labelled data, where each example has a known answer, and the model learns to predict that answer for new data. For example, in one of my projects, the portfolio agent learns allocation decisions from market history and gets a reward for the outcome. In a manufacturing setting, I would use supervised learning to predict whether a machine will fail in the next week, using past sensor readings labelled as failure or no failure.

Unsupervised learning has no labels. The model looks for structure on its own. For example, I would cluster machines by their operating patterns to find groups that behave similarly, and then compare them to find outliers. The main difference in evaluation is that supervised models can be scored against known answers, while unsupervised results need business interpretation and stability checks.

Q3. A client has 50,000 rows of tabular sales data. How would you choose between logistic regression and a tree-based model?

I'd start with a simple baseline, usually logistic regression for classification, so I have a reference point. Then I'd try a tree-based model like random forest or gradient boosting, because tabular data often has non-linear effects and interactions that trees capture well. The deciding factors are the client's needs. If they need to explain each factor's effect, for example to a sales manager, I'd lean towards logistic regression, since its coefficients are easy to explain. If accuracy matters more and the client accepts a less direct explanation, I'd choose the tree-based model and explain it with feature importance or SHAP values. In both cases I'd compare them on the same cross-validated split before deciding.

Q4. Your classifier is 97% accurate, but the positive class is only 3% of the data. Is it good?

Not necessarily. If I predicted the negative class every time, I would also get 97% accuracy and catch no positives at all. So I'd check the confusion matrix, then precision, recall, and F1 for the positive class. For imbalanced data, I'd also look at the precision-recall curve and PR-AUC. My reference point would be a majority-class baseline, and I'd only call the model good if it clearly beats that on recall and precision at a threshold the business accepts.

Q5. What is overfitting, and how would you detect it in a real project?

Overfitting means the model learns noise in the training data, so it performs well on the training set but poorly on new data. I detect it by comparing training and validation scores. A large gap that grows as the model gets more complex is the typical sign. I also use learning curves and cross-validation to confirm it. In my portfolio project, for example, I checked performance on rolling walk-forward windows rather than trusting one period, because a model that works in one market regime can fail in the next. To reduce overfitting, I use regularization, simpler models, early stopping, and more data where possible.

Q6. Explain a p-value to a non-technical manager.

A p-value tells us how surprising our result would be if nothing real were happening. For example, if a new process made no difference, a p-value of 0.02 means there is only about a 2% chance of seeing a difference this large by luck alone. It does not tell us the probability that the process works, and it does not tell us how big or useful the effect is. So I always report the size of the effect alongside the p-value, and I make sure the result is meaningful for the business, not just statistically significant.

Q7. How do you handle missing values in a client dataset?

First I find out why the values are missing, because the reason changes the right approach. For a few random gaps in a numeric column, I'd fill with the median, which is less affected by outliers than the mean. For categorical columns, I'd use the most frequent value or an "unknown" category. When missingness itself could carry information, for example a sensor going silent before a fault, I add an indicator column so the model can learn from it. I always fit the imputer on the training data only, so no information from the test set leaks into training. I put the imputer inside a pipeline so the same steps apply in production.

Q8. How would you explain a model's results to a client who does not know machine learning?

I start with the business question and the result in plain numbers. For example, "the model flags 80% of the machines that will fail, and each flag takes one inspection." Then I show two or three real examples so they can see what the model does. Next I explain the main drivers in simple terms, such as "vibration and temperature trends matter most." I also state the limits clearly, including how often the model is wrong and what each type of mistake costs. In my Clickstream project, I did the same with Metabase dashboards, so the team could see conversion and active users without reading any code.

Q9. Describe a time your model or project failed. What did you learn?

In the Clickstream project, my first design processed everything as a continuous stream, which was more complex than the business needed. The Silver and Gold layers only needed to update every few minutes. I changed those steps to scheduled incremental runs using Spark's availableNow trigger, orchestrated by Airflow every two minutes. That removed an always-on Spark job that was doing little useful work between updates. The lesson for me was to match the processing mode to the latency the business actually needs, and to be clear about the trade-offs, not to default to the most advanced option.

Q10. What is your current Japanese or English level, and what are your salary expectations and availability for relocation?

My English is [your English level, for example fluent in professional communication]. For Japanese, I'm currently at [your JLPT level, or "I have started studying Japanese and am working towards N4"], and I'm committed to continuing with the company's language training. I'm ready to relocate once the visa process is complete, and my notice period is [your notice period, for example: I'm a student, so my availability is after my graduation in May 2027]. For salary, I'm looking for a range of [your expected range], and I'm open to discussing it based on the role and the responsibilities.

Part B: LexAI

Q1. Walk me through LexAI from a user's question to the final answer.

When a user asks a question, the backend first optionally generates a hypothetical answer using the language model, which I use to improve the search query. Then I run two searches in parallel. One is dense retrieval with FAISS, where I search using the embeddings of the question and of the hypothetical answer. The other is sparse retrieval with BM25, which matches exact keywords. I merge the results with Reciprocal Rank Fusion, using a constant of 50. The top candidates go to a CrossEncoder reranker, which is bge-reranker-v2-m3. The reranker keeps the top five chunks, and those chunks go to the language model with a grounding prompt that tells it to answer only from the provided context. The answer streams back to the React frontend over Server-Sent Events, with citations for each source.

Q2. Why did you choose RAG rather than fine-tuning a model?

The main reason is that legal answers need to be traceable to a source. With RAG, every answer is tied to specific retrieved passages, so I can show citations. Fine-tuning would teach the model patterns, but it would not give me reliable citations, and it would be expensive to update every time a statute or judgment changes. With RAG, updating the knowledge means re-indexing the documents. The grounding prompt also reduces made-up section numbers, which matters a lot in legal text.

Q3. Why hybrid retrieval with FAISS and BM25? What does RRF do?

Dense retrieval understands meaning, so it can match “penalty for stealing” to a passage about “punishment for theft.” But it can sometimes miss exact legal terms and section numbers. BM25 is strong at exact matches, but it misses paraphrases. Using both covers both failure modes. The scores from the two systems are on different scales, so I don’t add them. Reciprocal Rank Fusion works on ranks instead: each document gets a score of one divided by (k plus its rank) in each list, and these are summed. I use k equal to 50. A chunk that ranks well in both lists rises to the top, and no score normalization is needed.

Q4. Your benchmark recall@5 went from about 0.003 to about 0.19. What does that mean?

The benchmark measures whether the retrieved chunks cover the correct evidence, at the character-span level, across 1,171 queries from the privacy_qa and contractnli datasets. I ran two benchmark runs on the same queries. FAISS alone reached a recall@5 of about 0.003 to 0.005. Adding BM25 with RRF raised it to about 0.11. Adding the reranker raised it to about 0.19 in both runs, and nDCG@10 was about 0.195.

I want to be clear about the absolute numbers. A recall@5 of 0.19 means most of the relevant evidence is still not in the top five. The improvement is large relative to the baseline, and it is measured and repeatable. But the absolute number shows that retrieval is the main limit of the system, and that is where I would focus next, with better chunking, stronger embeddings, and more candidates before reranking.

Q5. Why use a CrossEncoder reranker? What does it cost?

A bi-encoder, like the embedding model I use for FAISS, encodes the question and each document separately. That is fast, so it works over the whole corpus. A CrossEncoder reads the question and a candidate passage together, so it can judge relevance more accurately, but it cannot be precomputed, so it is slower. I use it in two stages. Fast retrieval narrows the corpus to about 20 candidates, and then the CrossEncoder scores only those 20. The cost is extra latency per query, and that is the trade-off I accepted for better ranking.

Q6. What is HyDE, and when could it hurt?

HyDE stands for Hypothetical Document Embeddings. A user’s question is often phrased very differently from the statute text. So I ask the language model to write a short, declarative passage that would answer the question, written in the style of a legal document, and I search with that passage as well as with the original question. Both result lists go into the same RRF fusion, so the original question still contributes. HyDE can hurt if the model writes a confident but wrong passage, because it can pull retrieval towards the wrong sections. That is why I keep the original question in the mix, and why the system continues without HyDE if the call fails.

Q7. How did you chunk the documents? Why semantic chunking and a 220-character overlap?

I used semantic chunking. The chunker embeds consecutive sentences and finds breakpoints where the meaning shifts sharply, using the 90th percentile of distance between sentences as the threshold. A percentile threshold adapts to each document, so a dense statute and a narrative judgment get different breakpoints. I merge chunks that are under 400 characters into their neighbours, and I force-split anything over 1,800 characters so no chunk exceeds the context budget. The forced splits use a 220-character overlap, so a sentence cut at a boundary still has its context on both sides.

Q8. Your resume says Indian legal question answering, but the benchmarks used privacy_qa and contractnli. Why?

That is a fair question. The benchmarks are English datasets from privacy policies and contracts, in the LegalBench-RAG format. I used them to test the retrieval mechanics, like hybrid fusion and reranking, on a standard, labelled dataset with ground-truth spans. The product target is Indian legal material, which is the statutes, the Indian Penal Code and related acts, and Supreme Court judgments. I have not built a labelled benchmark on that corpus yet, so I don’t have a retrieval number for Indian legal text. The next step would be to create a golden set of questions with the correct source spans from those documents.

Q9. How do you measure whether the answers are faithful to the sources?

For the generation side, I wrote an evaluation script that measures faithfulness and answer quality against reference answers. Right now it uses a token-overlap approximation, not the official RAGAS library. It computes an F1 score between the generated answer tokens and the reference tokens, and the overlap between retrieved context and the reference. I chose this because it is offline, free, and fast enough to run as a regression check. But I want to be honest about its limit: a paraphrase that is correct can score low, and a wrong answer that copies words can score high. I have a RAGAS compatibility fix written, but the LLM-judge version is not active yet, and that is the next thing I’d add.

Q10. What happens when the answer is not in the documents?

The grounding prompt tells the model to answer only from the retrieved context and not to fabricate section numbers, case names, or citations. If the context does not contain the answer, the model is instructed to say so. Retrieval is the one step with no fallback, so if no context is found, the system gives an explicit message rather than an ungrounded answer. Conversation memory is limited to the last eight messages, so the prompt stays small and the retrieved legal context gets most of the space. I would still test refusal behaviour more thoroughly, because a prompt instruction is not a guarantee.

Part C: AI-Powered Portfolio Optimization Engine

Q1. Explain the project end to end. What does the system output, and how is it evaluated?

The system takes daily market data for about 40 assets, drawn from the Nifty 50 and S&P 500, over roughly five years. The features include momentum and beta measures over several lookback windows, which are scaled and aligned on dates common to all assets. The main model is a reinforcement learning agent. On each step it outputs a 40-dimensional vector, which a softmax converts into portfolio weights that sum to one. The reward is the log return of the portfolio minus a transaction cost of 0.1% of turnover. The output is a set of target weights for the next rebalance. Evaluation uses walk-forward backtesting, where I train on a window of 504 trading days, test on the next 126, and then move forward 21 days. I compare the results with equal-weight and Markowitz baselines, using Sharpe ratio, drawdown, and CVaR.

Q2. Why PPO and SAC? What is the difference between them?

PPO is an on-policy algorithm. It collects fresh experience with its current policy, and it limits how much the policy can change in one update using a clipping range of 0.2. That makes training stable. I used it as the primary agent, trained for 100,000 environment steps. SAC is off-policy. It reuses past experience from a replay buffer, and it adds an entropy term that rewards exploration. I used it as a challenger, trained for 150,000 steps. Comparing them showed me how much the result depends on the algorithm, and the MLflow registry lets me promote whichever model performs better on validation data.

Q3. How did you prevent look-ahead bias and survivorship bias?

For look-ahead bias, I made sure every step uses only data available at that point in time. The scaler is fitted on the training window only, so the test period never influences the scaling. The Markowitz covariance matrix uses a strictly backward-looking window of past returns before each rebalance date. Features are calculated using past data only.

Survivorship bias is harder. The asset universe is a fixed list of index constituents, and if that list reflects today’s members, the backtest excludes companies that failed or were removed in the past. So I have to be honest that this is a limitation of the current version. The fix is to use point-in-time index membership, so each rebalance date only includes the stocks that were actually in the index at that time. [Confirm how your ticker list was built before the interview.]

Q4. Explain walk-forward validation. Why not one train and test split?

Walk-forward validation trains on a window, tests on the period right after it, and then slides the whole window forward. In my setup, each window has 504 days for training, 126 days for testing, and moves forward 21 days each time. That gives around 11 to 30 test windows, depending on how much history is available. A single split gives one estimate, and that estimate depends on which market period it happens to cover. Walk-forward shows how the strategy performs across many periods, including bull, bear, and sideways markets, which is much closer to how it would be used in practice.

Q5. Your Sharpe ratio is 1.4 versus 0.6 for the benchmark. What does Sharpe measure, and would the result hold after trading costs?

Sharpe ratio is the average excess return divided by the standard deviation of returns, annualized by the square root of 252. In my walk-forward results, the RL agent had a Sharpe of about 1.4 to 1.47, Markowitz about 0.74 to 0.77, and the Nifty 50 benchmark about 0.61. Sharpe has limitations. It treats upside and downside volatility the same, and it does not capture tail risk, which is why I also report maximum drawdown and CVaR. On costs, the training reward already subtracts 0.1% of turnover, so the agent learns to trade only when the benefit is worth the cost. I would still want to test higher cost assumptions and check how much turnover the strategy creates before any real use.

Q6. Explain Markowitz and Black-Litterman. Why is mean-variance optimisation unstable?

Markowitz mean-variance optimisation chooses weights that balance expected return against variance, using the covariance matrix of asset returns. It is very sensitive to the inputs. Small errors in expected returns can produce very concentrated portfolios, and the covariance estimate is noisy when there are many assets relative to the number of observations. I have 40 assets and 504 observations per window, so the raw sample covariance is unreliable. I used Ledoit-Wolf shrinkage, which blends the sample covariance with a structured target to produce a more stable estimate. [Black-Litterman: confirm whether it is implemented in your code. If it is not, say you included it as a conceptual comparison, and that it blends market equilibrium returns with your own views to reduce this sensitivity.]

Q7. You report statistical significance across rolling windows. Which test did you use?

I ran a two-sided one-sample t-test on monthly excess returns, meaning the RL strategy's return minus equal-weight's return, aggregated by month across all windows. The null hypothesis is that the average excess return is zero. The README reports a p-value of about 0.024, and a different, larger run reported a much smaller p-value. Both are below 0.05. I want to be clear that the monthly returns are not fully independent, and I did not apply a formal correction for testing many windows. So I present this as supporting evidence, not as proof of a real edge. I would validate it with a larger out-of-sample period before relying on it.

Q8. How is the reward defined, and what are the risks of using reinforcement learning for trading?

The per-step reward is the log return of the portfolio minus 0.1% of turnover. I use log return because it adds up across time. The cost term stops the agent from rebalancing constantly. For evaluation of whole episodes, I use Sortino ratio, because it penalizes only downside volatility. Using Sharpe as the reward could push the agent to avoid large positive days, which is not what we want, so Sortino is the better choice for episode scoring. The risks with RL in trading are overfitting to one market regime, sensitivity to the reward design, unrealistic execution assumptions, and training instability. So I monitor drawdown and turnover, not only the headline return.

Q9. How do you detect model drift and decide when to retrain?

I track the rolling 30-day Sharpe ratio of the live strategy. If it falls below 0.5, the system raises a drift alert. For model promotion, I use a champion-challenger rule. A newly trained model becomes the champion only if its validation Sharpe is at least 5% higher than the current champion's. That stops me replacing a good model because of small noise. The models are stored in the MLflow registry with a "champion" alias, and the serving layer loads whichever version holds that alias. The retraining runs nightly through Airflow. I would add input distribution monitoring as well, because a drop in Sharpe is a late signal.

Q10. Why does Airflow only call HTTP endpoints, and how do you prevent two training jobs from running at once?

I kept Airflow as a thin orchestrator. Its DAG has no ML code. Each task makes an HTTP call to the FastAPI service, which runs the actual data processing and training. That keeps Airflow's environment small, and a crash in a training job does not take down the scheduler. The DAG fetches data, engineers features, trains the PPO and SAC models in parallel, evaluates the champion, and sends a notification. To prevent overlapping training, the service uses a training-in-progress flag guarded by a lock, so a second training request gets a conflict response instead of starting. I would add a timeout so that a crashed job cannot hold the lock forever.

Part D: Real-Time E-Commerce Clickstream Lakehouse

Q1. Walk me through the architecture from Kafka to the dashboard. Why a medallion design?

A Python event generator produces view, add-to-cart, purchase, and remove-from-cart events as JSON, and sends them to a Kafka topic called ecommerce.clickstream. Spark Structured Streaming reads that topic and writes three Delta Lake layers. Bronze stores the raw events, after parsing and validation, with malformed records sent to a quarantine path. Silver removes duplicates, enriches the events, and enforces mandatory fields. Gold computes business aggregates such as conversion rate, cart abandonment, and active shoppers in tumbling windows. From Gold, a sync job writes to PostgreSQL, where dbt builds a star schema with dimension tables for users, products, and time, and a fact table of user sessions. Metabase reads from PostgreSQL to show dashboards. I used the medallion design because each layer has a different job. Keeping Bronze raw means I can always replay history if a bug is found downstream.

Q2. Why key messages by user_id, and why three partitions?

Kafka guarantees order only within a partition. Keying by user_id sends all of one user's events to the same partition, so their events stay in order, and sessionization depends on that order. Three partitions let three consumers read in parallel. The trade-off is that more partitions give more parallelism, but they make rebalancing and ordering across partitions more complex. For this project's scale, three was a sensible starting point, and I created the topic with that count up front, because changing the partition count later affects key-to-partition mapping.

Q3. What is a watermark? Why 10 minutes, and what happens to events that arrive later?

A watermark tells Spark how long to wait for late events before it treats a window as complete. I set it to 10 minutes. The producer deliberately sends about 5% of events up to 10 minutes late, so the pipeline has real late data to handle. Events that arrive within the watermark are included in their window. Events that arrive after the watermark are dropped from the windowed state, so state does not grow forever. The trade-off is correctness against latency. A longer watermark catches more late events but holds results back for longer, and it keeps more state in memory.

Q4. How do you handle duplicates? Is your pipeline exactly-once?

No, the pipeline is at-least-once, and I don't claim exactly-once. Kafka with acks equal to all and retries can redeliver messages, so duplicates can happen. I handle them at each layer. The producer keys by user_id, which keeps per-user ordering. Silver removes duplicates with dropDuplicatesWithinWatermark on event ID and timestamp, so the same event is not stored twice. Gold uses Delta MERGE, so reprocessing a window replaces its row instead of adding a new one. The Postgres writes use ON CONFLICT, so repeated writes are skipped or updated, not duplicated. Together, reprocessing the same data gives the same result, which is what idempotent means here. Exactly-once end to end was out of scope for this project, and I documented it as a limitation.

Q5. Why Delta Lake instead of plain Parquet? Why use MERGE for the Gold layer?

Delta adds a transaction log, called _delta_log, on top of Parquet files. That gives me ACID writes, so a failed job does not leave a half-written table, and it gives me schema enforcement and time travel. For Gold, the aggregates change as more late events arrive for the same window. MERGE lets me update the existing row for that window or insert a new one in one atomic operation. A plain Parquet append would create duplicate rows for the same window every time it was reprocessed.

Q6. You orchestrate with Airflow every two minutes. Is this really real-time?

I want to be precise. Kafka ingestion into Bronze runs continuously, so events land within seconds. But the Silver, Gold, and Postgres sync steps run as scheduled incremental batches every two minutes, using Spark's availableNow trigger. So the end-to-end latency to the dashboard is minutes, not seconds. I made that choice deliberately. Keeping an always-on Spark job for steps that only need to run every two minutes wastes resources. If the business needed sub-second dashboards, I would move Gold to a continuous trigger and serve from a store designed for low-latency reads.

Q7. The DockerOperator needs the Docker socket. What is the risk?

Mounting the Docker socket into a container gives that container control over the Docker daemon, and that is effectively root-level control of the host. A compromised Airflow task could start a privileged container that mounts the host filesystem. I know this is a local-development trade-off, and my documentation calls it out. In a real deployment, I would use KubernetesPodOperator, which runs tasks as pods without the socket, or run Spark jobs through spark-submit on a managed cluster, with tightly restricted worker permissions.

Q8. How do you check freshness and data quality?

For freshness, the Airflow DAG starts with a gate task. It reads the modification times of the JSON commit files in Bronze's _delta_log directory, and fails if the latest commit is older than a set staleness threshold. That tells me ingestion has stopped without starting a Spark session just to check. For quality, dbt tests run on the warehouse models. I test that keys are unique and not null, and that every user_id and product_id in the fact table exists in its dimension table. If a test fails, dbt reports it in the test output, so the problem is visible rather than hidden.

Q9. Walk me through your star schema. What is the grain of the fact table?

The fact table, fact_user_sessions, has one row per session. Every column is an aggregate computed by grouping the staging events by session ID. The dimensions are dim_users, dim_products, and dim_time. Getting the grain right matters, because if I accidentally joined at the event level, the session counts would be inflated. I check this with a test that compares the row count of the fact table with the count of distinct session IDs. dim_time is built from the events that exist, so an hour with no events simply has no row. I noted that a full calendar dimension would be the next step if reports needed empty periods.

Q10. How did you handle bad records and schema changes?

In Bronze, each incoming message is parsed against an explicit schema. If parsing fails, or a required field is missing, or the event type is unknown, the record goes to a separate quarantine path with its own checkpoint. Nothing is silently dropped, and the main stream keeps running. I can inspect the quarantined records and see exactly why they were rejected. For schema changes, Delta enforces the schema on writes, so an unexpected change fails loudly instead of corrupting the table. Adding a new field would be a deliberate schema change in the producer and the parser, not something the pipeline absorbs silently.

Items in [square brackets] need your own details or a check of your code before the interview.